Unidad 3: Gestión de Proyectos de Software

Semana 4: Técnicas de planificación y gestión

- Tareas, tiempos y recursos
- Introducción a diagramas de Gantt y cronogramas
- Concepto de entregables y control de avance

Roadmap

"Los planes no valen nada, pero planear es esencial "

Con esta frase comenzamos nuestra unidad de Gestión de proyectos, el roadmap es una herramienta que tiene por objetivo dar visibilidad a alto nivel de nuestra estrategia para la evolución de un producto a lo largo del tiempo según la información actual. Este pronóstico de como el producto puede evolucionar permite tener una sensación de continuidad y propósito; ayudando a priorizar decisiones , calmando ansiedades y facilitando la organización de la relación del producto con su contexto. De esta manera se permite a los involucrados (directo e indirectos) estar alineados y negociar por los recursos requeridos para realizar ese camino.

Foco en el ¿Para qué?

El "para qué" te lleva a centrarte en el propósito, no en el formato.
No se trata solo de dibujar un cronograma bonito o de listar tareas.
Se trata de que el roadmap sirva como herramienta de comunicación y alineación estratégica.
Por ejemplo:

Para qué: Alinear al equipo sobre la visión del producto.
Para qué: Mostrar a los stakeholders cómo evolucionará el proyecto.
Para qué: Priorizar decisiones y recursos según la estrategia de negocio.

El **formato del roadmap puede variar** (Gantt, flechas, líneas, etc.), pero lo importante no es **cómo se ve**, sino **para qué se usa**.
El foco no debe ser en "hacer un roadmap porque hay que hacerlo", sino en **qué valor aporta** al equipo o al producto.

Aspecto	Enfocarse en el "Qué" o "Cómo"	Enfocarse en el "Para qué"
Objetivo	Mostrar tareas y tiempos	Mostrar dirección, propósito y visión
Resultado	Cronograma operativo	Herramienta estratégica
Beneficio	Organización interna	Alineación y toma de decisiones
Ejemplo	"Implementar módulo X en noviembre"	"Entregar valor al usuario mediante el nuevo módulo X antes del cierre del año fiscal"

¿Qué tan extenso?

Cuando alguien pregunta *"¿qué tan largo o detallado tiene que ser un roadmap?"*, **no hay una respuesta única**.
Depende de varios factores:

- quién lo va a usar (la audiencia),
- para qué se necesita (el objetivo),
- en qué etapa está el producto o el proyecto,
- y cuánto se sabe del futuro (porque a veces hay más certezas, y otras veces, más incertidumbre).

Un roadmap puede ser **más general o más detallado** según la situación.

Largo plazo

- Significa planificar a más de un año.
- Sirve para **visualizar los grandes pasos o hitos** que el producto o proyecto tiene que alcanzar en ese tiempo.
- Te ayuda a conectar la estrategia general del negocio con el desarrollo del producto.

Planificar a largo plazo **ayuda a anticipar acuerdos y recursos** que pueden tardar en conseguirse (como contratar proveedores, cerrar alianzas o conseguir presupuestos).

Roadmap cuatrimestral con el cual Google rediseño su producto Calendar en el año 2017

Corto plazo

Un **roadmap de corto plazo** es una planificación visual que muestra **las acciones, entregas o hitos más próximos**, normalmente dentro de un **período de hasta un año** (por lo general, entre 3 y 6 meses).

Su objetivo es **guiar la ejecución táctica**: qué se va a hacer ahora, qué está en curso y qué viene inmediatamente después.

Características principales:

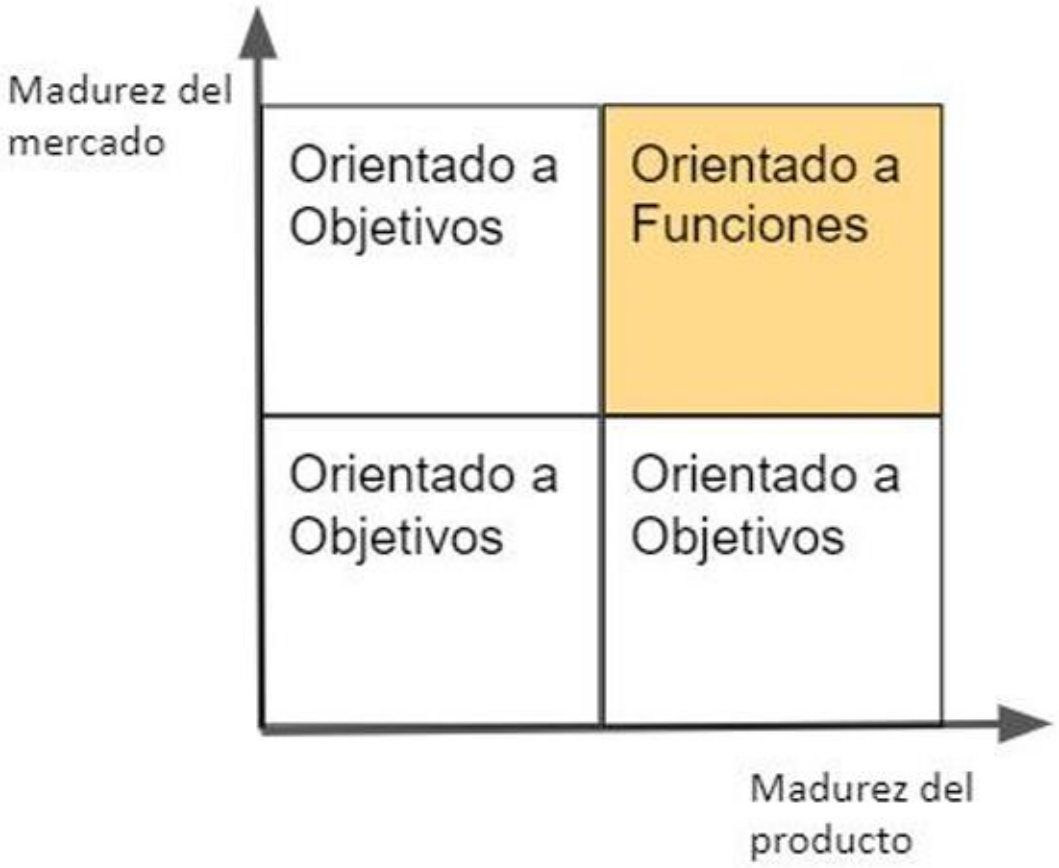
- Se enfoca en **lo operativo y concreto**.
- Incluye **fechas más precisas** y **tareas detalladas**.
- Se actualiza con frecuencia (por ejemplo, en cada sprint o trimestre).
- Permite **ajustar el rumbo rápidamente** si cambian las prioridades o surgen nuevos datos.

Ejemplo:

Un roadmap trimestral (Q1 – Q2 – Q3) que detalla las funcionalidades a desarrollar, las pruebas planificadas y los lanzamientos inmediatos.

En agilidad, este tipo de roadmap es el más usado, porque **acompaña el ritmo iterativo y adaptable** del trabajo.

Tipos de Roadmap



Como sus nombres pueden sugerir, buscan ordenarse por funcionalidad o por objetivo. Podríamos identificar por funcionalidad tales como: “permitir pagos con tarjeta de crédito”, “envio a domicilio” mientras que los otros indican objetivo de valor como: “aumentar margen de ganancia”, “lograr un porcentaje de un mercado”

En los orientados a funcionalidades debe estar alineada con la visión y estrategia definidas para el producto. Este es el más utilizado en las organizaciones tradicionales como única opción. Si bien suena intuitivo y coherente, resulta restrictivo para la experimentación/búsqueda de opciones de implementación ya que queda establecido de un comienzo el QUE y comúnmente también el COMO. Por tal motivo este tipo de roadmap es una herramienta para entornos mas estables en dominios conocidos.

A diferencia de cuando expresamos objetivos buscados en un intervalo de tiempo, dejamos abierto COMO lograrlo, pudiendo validar diversas hipótesis y sobre todo conservando capacidad de cambios por incertidumbre complejidad u otros motivos

Creando un roadmap orientado a objetivos

Vamos a elaborar el roadmap de un producto ficticio: una tienda en línea de libros usados.

Vamos a plantear la estrategia considerando que el producto aún no sale al mercado y que necesitamos documentar la estrategia del producto para los próximos doce meses.

Paso 1. Dividimos el año en periodos que nos permitirán separar en etapas la estrategia general del producto. Debido a que se trata de una visión general, es recomendable hacer la división en meses o trimestres, dependiendo del tipo de producto que tengas.

DATE The release date or timeframe	Date or timeframe	Date or timeframe	Date or timeframe	Date or timeframe
	Q1	Q2	Q3	Q4

Paso 1: Definir periodos

Paso 2. En un roadmap orientado a objetivos lo más importante es definir un objetivo para cada periodo. Los objetivos deben estar alineados con la visión del producto y el objetivo general.

NAME The name of the new release	Name/version	Name/version	Name/version	Name/version
	Lanzamiento Tienda Libros	Tienda Libros	Tienda Libros 1.2	Tienda Libros 1.3

Paso 2: Darle nombre a las etapas

Paso 3: Definimos cómo vamos a medir el éxito de nuestra meta u objetivo. Esta parte es fundamental para conocer el avance del proyecto en cada periodo. Si no estás seguro de cómo medir tu objetivo quizá debas revisar si el objetivo está planteado correctamente.

GOAL The reason for creating the new release	Goal	Goal	Goal	Goal
	Desarrollo y lanzamiento de tienda online funcional con un catálogo de 50 productos	Captar el correo electrónico del 5% de los usuarios que visitan la tienda.	Lograr 100 ventas mensuales	Obtener un 100% de clientes satisfechos con su compra

Paso 3: Definición de los objetivos

Paso 4: Elaboramos una lista de funcionalidades deseadas que nos ayudarán a cumplir el objetivo planteado para cada trimestre. Esta lista debe ser muy general, sin detalles ni especificaciones y debe estar alineada con los pasos anteriores.

FEATURES The high-level features necessary to meet the goal	Features	Features	Features	Features
	*Diseño e implementación de la UX *Desarrollo del proceso de pago con tarjeta	*Integración de proceso de captación de leads.	*Blog (contenido) *Optimización del proceso de compra *Integración email	*Cuestionario de satisfacción *Chat en línea

Paso 4: Definición de las funcionalidades

Paso 5. Asignamos un nombre para identificar las etapas del producto. Para productos digitales es común manejarlo en versiones o *releases.*

Paso 5: Selección de métricas

Es importante mencionar que no precisamente esta será la versión final del roadmap. Lo más probable es que sufra alguna o muchas modificaciones en el transcurso del año.

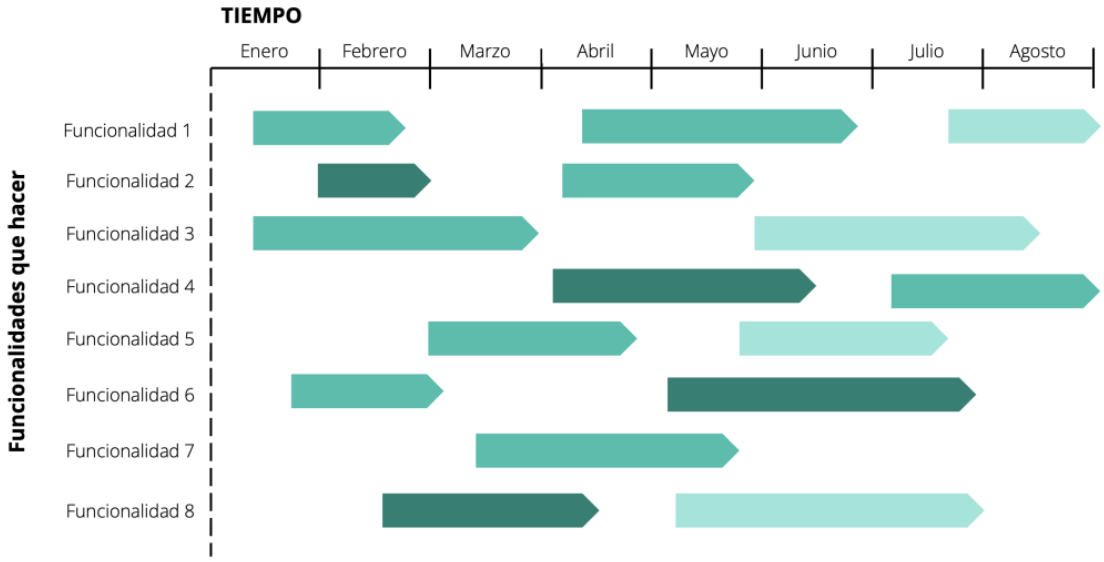
Se debe ajustar las métricas o cambiar completamente la estrategia si las cosas no están yendo como lo esperado.

Recuerda que la finalidad del roadmap es plantear una estrategia global del producto y comunicarla correctamente para alinear los esfuerzos de cada equipo.

Creando un Roadmap orientado a funcionalidad

Un roadmap orientado a funcionalidad podría visualizarse en un tablero con las siguientes columnas, organizadas por tiempo o prioridad:

- **Ahora (Now)**: Funcionalidades en desarrollo activo.
- **A continuación (Next)**: Funciones prioritarias que se abordarán después de las actuales.
- **Más tarde (Later)**: Funciones planificadas a medio o largo plazo.



¿Qué NO es un product *roadmap*?

Un roadmap no es un mapa de funcionalidades con fecha de liberación.

No es un *checklist* con las actividades por realizar.

No es una guía táctica con tareas específicas y definidas.

Un roadmap debe contener únicamente los elementos clave que permitan a cualquier colaborador de tu equipo (gerente, programador, proveedor, etc.) entender qué es y hacia dónde se dirige el producto.

Principales características de cada tipo de formato de Roadmap:

Agile Roadmap	TimeLine Roadmap
– Más flexibles – Horizonte temporal más amplio – Centrado en el resultado esperado	– Más estrictos y rígidos – Plazos establecidos y conocidos – Centrado en tareas ya establecidas

Diagrama de Gantt

No conviertas tu Roadmap en un diagrama de Gantt, ya se parece bastante no termines transformándolo. Veamos la diferencia:

- En un diagrama de Gantt (gestión de proyectos) consiste en enumerar un conjunto de tareas, indicar el alcance (fechas), y averiguar cuál es la mejor manera de planificarlo para garantizar que se realiza en un plazo establecido y con un costo razonable.

- En un Roadmap de Producto (gestión de productos) tiene una visión mucho más amplia, no se planifican tareas, se construye un plan para resolver **problemas** que tiene el usuario o la compañía. En la gestión de productos no se conoce el alcance total del proyecto. ¿Por qué? porque la idea es ir aprendiendo e iterando nuestro producto en función de las necesidades del mercado. Con esto conseguimos seleccionar los proyectos que llevarán al éxito en el futuro.

El **diagrama de Gantt** es una **herramienta de planificación detallada**, que muestra **tareas, duraciones, dependencias y fechas concretas**.

"¿Quién hace qué y cuándo?"

Se usa para:

- Gestionar **tiempos, recursos y dependencias**.
- Controlar el **avance real** del proyecto.
- Coordinar el **trabajo entre distintos equipos**.

Ejemplo:

Semana 1–2: Relevamiento → Semana 3–4: Diseño → Semana 5–8: Desarrollo → Semana 9–10: Pruebas.

Aspecto	Roadmap	Diagrama de Gantt
Propósito	Comunicar la **visión y dirección** general	Planificar y **controlar la ejecución** detallada
Nivel de detalle	Bajo / estratégico	Alto / operativo
Enfoque	Qué se logrará y en qué orden	Qué tareas se hacen, cuándo y por quién
Horizonte temporal	Mediano o largo plazo (meses o años)	Corto o mediano plazo (semanas o meses)
Actualización	Ocasional (cuando cambia la estrategia)	Frecuente (cada cambio en el cronograma)
Audiencia	Directivos, stakeholders, equipo general	Jefes de proyecto, desarrolladores, QA, etc.
Formato típico	Fases o hitos (sin fechas exactas)	Tareas con fechas precisas y barras de duración

Roadmap = visión estratégica → "qué haremos y hacia dónde vamos".
Gantt = ejecución operativa → "cómo y cuándo lo haremos".

¿Para qué se estima?

Estimamos para **poder tomar decisiones informadas**.
Por ejemplo:

- Decidir si vale la pena hacer un proyecto.
- Planificar el trabajo y los recursos.
- Medir si estamos cumpliendo con lo esperado.

En los **enfoques ágiles**, lo importante no es estimar por obligación, sino preguntarnos **qué valor nos aporta esa estimación**.
Si realmente ayuda a decidir, organizar o mejorar, entonces vale la pena hacerla.

¿Cuándo se estima?

Se puede estimar en diferentes momentos, según la necesidad:

1. **Antes de elegir un proyecto:**
   Para comparar opciones y decidir **cuál conviene más** según el valor o esfuerzo requerido.
2. **Al iniciar un proyecto:**
   Para armar un **roadmap** (una hoja de ruta general) y coordinar recursos, equipos y tiempos.
3. **Al comenzar una etapa:**
   Para planificar en detalle qué se hará, con qué recursos y en qué orden.
4. **Día a día:**
   Para ajustar las tareas según las prioridades o imprevistos del día y ver cómo se conectan entre sí.

Costo de la estimación

Hacer una estimación **consume tiempo y esfuerzo** del equipo, por lo tanto **tiene un costo**. No genera directamente un producto nuevo, sino que es un trabajo interno para planificar mejor.

Por eso:

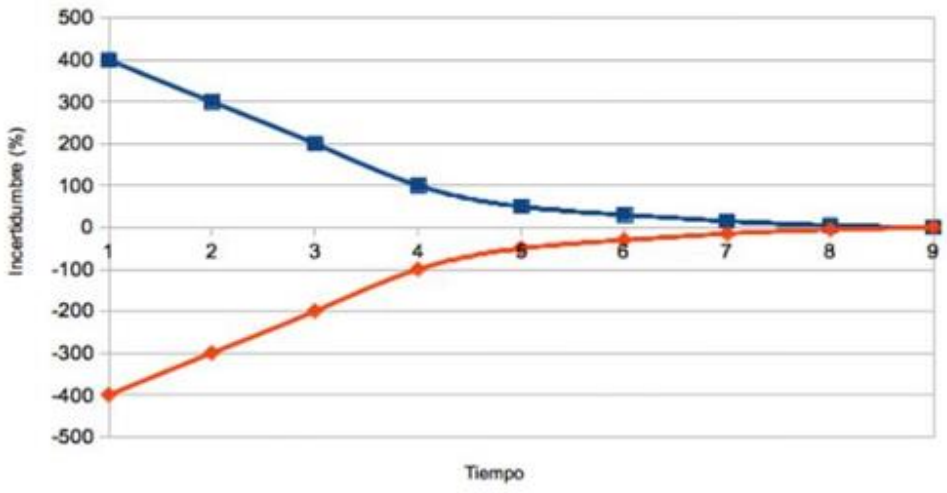
- No se trata de **presionar al equipo** con la estimación, sino de usarla como una herramienta útil.
- Antes de estimar, conviene preguntarse:
    - ¿Qué queremos validar con esta estimación?
    - ¿Qué valor tiene para el cliente o el equipo?

Riesgo o incertidumbre al estimar

Al principio de un proyecto, **sabemos poco**, por lo tanto las estimaciones son **menos precisas**. A medida que avanzamos y tenemos más información, las estimaciones se vuelven **más exactas**.

Esto se representa con el concepto de **"cono de la incertidumbre"** de Barry Boehm:

Al inicio hay mucha incertidumbre, pero se va reduciendo con el tiempo y la experiencia.



Cono de Incertidumbre – Barry Boehm

Al **inicio del proyecto** hay **mucha incertidumbre**: se sabe poco y las estimaciones pueden estar muy lejos de la realidad.

Con el tiempo, al avanzar en el proyecto y obtener más información, las estimaciones se vuelven **más precisas**.

Por eso, **las decisiones críticas o compromisos grandes** (como presupuesto o fechas finales) **conviene tomarlas más adelante**, cuando la incertidumbre es menor.

En los **entornos ágiles**, como Scrum, **no se busca estimar en horas o días exactos**, sino de forma **relativa**: comparando tareas entre sí.
Esto se llama **estimación relativa**, y es una práctica recomendada por autores como:

- **Mike Cohn** (autor de *Agile Estimating and Planning*)
- **Ken Schwaber** y **Jeff Sutherland** (creadores de Scrum)
- **Barry Boehm**, que estudió el *cono de incertidumbre* que explica por qué al inicio las estimaciones son poco precisas

¿Qué significa "estimación relativa"?

Significa **comparar el esfuerzo o complejidad de una tarea con otras tareas conocidas**, en lugar de calcular tiempo real.

Por ejemplo:

- Si una historia de usuario A es el doble de compleja que la historia B,
  → A vale "2 puntos" y B vale "1 punto".
  No estamos diciendo que A tarda 2 días, sino que **requiere el doble de esfuerzo que B.**

¿Por qué "relativa" y no "absoluta"?

Porque al inicio de un proyecto (como explica Boehm con el cono de incertidumbre) **no tenemos suficiente información para estimar con exactitud**.
Por eso, **comparar es más confiable que adivinar tiempos exactos**.

En vez de decir:

"Esto llevará 5 horas"

se dice:

"Esto es más grande que la tarea X, pero más chico que la tarea Y".

Autor / Fuente	**Aporte**
Mike Cohn (2005)	Introduce la práctica de *Story Points* (puntos de historia) y *Planning Poker* como formas de estimación relativa.
Jeff Sutherland & Ken Schwaber (Scrum Guide)	Proponen usar estimaciones relativas en backlog refinements para medir esfuerzo y capacidad del equipo.
Barry Boehm (Cono de Incertidumbre)	Demuestra que las estimaciones iniciales tienen grandes errores, justificando el uso de valores relativos en lugar de tiempos fijos.

Herramientas comunes para estimación relativa

1. **Story Points (Puntos de historia)**
   → Miden *esfuerzo relativo*, *complejidad* y *riesgo*.
   Escala más usada: **1, 2, 3, 5, 8, 13…** (Serie de Fibonacci).

2. **Planning Poker**
   → Técnica colaborativa: cada miembro del equipo elige una carta con su estimación, se comparan y discuten diferencias hasta llegar a consenso.

3. **T-shirt Sizes (Tamaños de remera)**
   → Clasificación simple: XS, S, M, L, XL (según esfuerzo relativo).
   Muy útil en etapas tempranas de un proyecto.

Enfoque tradicional	**Enfoque ágil**
Se estima en horas o días.	Se estima en puntos o tamaños relativos.
Alta incertidumbre al inicio.	Se reduce el error al comparar.
Enfocado en control y cumplimiento.	Enfocado en aprendizaje y valor entregado.

Ventajas de la estimación relativa

- Reduce discusiones sobre “cuántas horas” lleva algo.
- Ayuda a enfocar al equipo en el valor del producto, no en el control del tiempo.
- Se adapta mejor al aprendizaje progresivo (conforme baja la incertidumbre).
- Permite predecir la velocidad del equipo (cuántos puntos completan por sprint).

Concepto de entregables y control de avance

Para poder entender lo que se quiere obtener con un entregable, les dejo esta frase

"Cada iteración debe entregar software funcionando, no promesas ni planes."

El entregable central es el Incremento:
"la suma de todos los Product Backlog Items completados durante un Sprint".

Cada incremento debe estar en condiciones de ser liberado (cumple con la *Definition of Done*).

"El Incremento es un paso concreto hacia la visión del producto."

En referencia al control de avance les dejo dos frases para que puedan reflexionar

"El progreso se controla midiendo cuánto valor se entrega, no cuánto trabajo se hace."

El control no se logra por supervisión, sino por inspección, adaptación y transparencia.

- El avance no se mide por tareas completadas, sino por funcionalidades entregadas (historias de usuario terminadas y probadas).
- Se usa el Burndown Chart o Burnup Chart para visualizar progreso real del sprint o del proyecto.
- También propone medir la velocidad del equipo (velocity): cantidad de puntos de historia completados por iteración.

Autor	Qué es un entregable	Cómo se controla el avance
Mike Cohn	Software funcional o historia de usuario terminada	Burndown charts y velocidad del equipo
Schwaber & Sutherland	Incremento listo para entregar (Definition of Done)	Transparencia + Daily + Sprint Review
Boehm	Entregas que reducen la incertidumbre	Comparación entre estimado y real; aprendizaje iterativo
Kent Beck / XP	Código funcionando y probado	Pruebas automáticas, feedback continuo

En los enfoques ágiles, el entregable es valor real para el usuario, y el control de avance se logra a través de visibilidad, feedback y adaptación continua, no por reportes ni control de tiempo.

Definición de terminado (done)

La imagen muestra **las distintas etapas de validación y prueba** que atraviesa un software durante su desarrollo. Cada rectángulo representa un **nivel de feedback (retroalimentación)**, desde los más tempranos (al desarrollar) hasta los más tardíos (ya en producción).

La **flecha roja** indica que **a medida que se avanza hacia etapas posteriores**, el **"costo del error" aumenta exponencialmente**.

- Muchas veces sucede que la comunicación no es clara cuando se quiere definir que es terminado. Por ejemplo, cuándo un usuario especifica que requiere un reporte con cuatro columnas y que la última columna tenga un total parece algo sencillo. Pero si para probar ese reporte se requiere preparar datos históricos consolidados de varios meses y años entregados por un proveedor externo, además cuadrar los resultados con otros reportes y además que dichos reportes sean validados con datos de mínimo 6 meses, la cosa se complica bastante y los conflictos suelen aparecer con frecuencia más aún cuando el único punto de referencia es un documento de análisis donde no aparece si esto se encuentra dentro o fuera del alcance y por lo tanto el único entregable es el "Reporte terminado".

- También sucede que frecuentemente cuándo un miembro del equipo de desarrollo se refiere a que ha terminado una funcionalidad se refiere al término de la codificación y no al termino de dicha funcionalidad como un incremento de producto listo para ser usado para el usuario. Esto lleva muchas veces a conflictos porque se puede suponer que hay una mala intención del desarrollador al entregar un producto incompleto.

- En otras ocasiones supuestamente se ha terminado de desarrollar una aplicación con las pruebas del producto y revisado con los usuarios pero cuando se intenta pasar a producción los usuarios mencionan que requieren seguir probando con otros meses y además que hay otras integraciones que se requieren probar y que sin esas pruebas no pueden aprobar el producto, esto en realidad nunca estuvo en el plan de trabajo pero dado que el usuario siempre recalca que mientras no tenga el "producto terminado" no lo va aceptar no tenemos claro cuando hemos terminado, pero ¿Qué es terminado?

- La definición de "Terminado" es un entendimiento compartido de completado para el producto.

- Debe ser visible y universalmente entendido y acordado para fomentar la transparencia.

- Es el denominador común de la calidad del producto.

- Por ejemplo, un equipo de desarrollo que entrega una funcionalidad al Product Owner cree que ya ha terminado, el Product Owner cree que eso significa que junto con esa funcionalidad se ha elaborado la documentación asociada a la funcionalidad. El equipo de desarrollo no ha entregado esa documentación y no entiende porque se requería. Es un clásico problema de comunicación que podría ser resuelto por la definición de terminado.

La deuda técnica provoca una crisis en la profesión del desarrollo de Software.

- Una creencia de algunos clientes es que pueden pedir software y exigir fechas y alcances definidos y que ese software puede ser entregado en un estado terminado y con todos los requerimientos completados.

- Los desarrolladores para soportar esa creencia tienden a reducir la calidad para cumplir con las fechas y el alcance comprometido.

Entonces se producen resultados como los siguientes

- Software con errores, empresas que pierden dinero, clientes insatisfechos y que cada vez creen menos en la profesión del desarrollo de software.

- Los desarrolladores llegan a sentirse desgastados, desmotivados y con un sentimiento de resentimiento y disconformidad con la profesión del desarrollo de software. Pueden llegar hasta odiar el trabajo porqué a pesar de todo el esfuerzo no logran entregar productos de software “terminado”.

- Se produce un largo, tedioso y complejo proceso de estabilización que desgasta las relaciones entre el equipo de desarrollo y los usuarios.

Conforme el trabajo no terminado crece el trabajo terminado decrece creando un escenario en el cuál se invierte más tiempo en tratar de pagar la deuda técnica que en desarrollar más producto terminado. Se puede llegar a un momento en el cual la deuda técnica es tan grande

que casi no se desarrolla nuevo producto y todo el tiempo se invierte en pagar esa deuda técnica.

¿Qué debería incluir la definición de terminado?

- Pruebas de rendimiento
- Pruebas de escalabilidad
- Pruebas de seguridad
- Calidad de código (Por ejemplo deuda técnica medida con SonarQube o usar NDepends)
- Refactoring
- Pruebas de aceptación del usuario
- Documentación requerida por la organización
- Pruebas de regresión
- Revisión de código
- Pruebas de integración

Los criterios de terminado en Scrum proveen transparencia

- Tener una definición de terminado completo para el desarrollo que sea similar al producto en producción
- Asegurarse que la definición de terminado optimice el mantenimiento y extensibilidad de la aplicación
- En cada Sprint cumplir los criterios de terminado y refinar estos criterios durante el Sprint Retrospective
- Cumplir los criterios de terminado por elemento del Sprint Backlog del Sprint para evitar sorpresas al finalizar.

1. **Validación de Arquitectura**
    - Es el nivel más básico.
    - Se verifica que la arquitectura técnica esté bien diseñada antes de comenzar a desarrollar.
    - Si se detecta un error aquí, **corregirlo es muy barato**, porque aún no hay código.
2. **Unit Test, BDD (Behavior Driven Development)**

- Son las pruebas unitarias automáticas y de comportamiento.
- Validan que cada parte del código funcione correctamente por separado.
- Los errores se detectan rápido, con bajo costo.

3. **Pruebas del desarrollador en verde**

   - El desarrollador prueba su propio código y confirma que "todo pasa en verde".
   - Se asegura que el módulo individual funcione antes de integrarlo.

4. **Check-In, Branches, Labels**

   - Momento en que se integra el código al repositorio (control de versiones).
   - Aquí se revisa que el código sea consistente con el resto del proyecto.
   - Los errores detectados aquí ya tienen un costo mayor (pueden afectar a otros módulos).

5. **Integración, Despliegue y Entrega Continua (CI/CD)**

   - Se integran todos los componentes del sistema.
   - Se automatiza el proceso de construcción, prueba y entrega del software.
   - Si hay errores, el impacto ya es más amplio.

6. **Pruebas de Integración y de Regresión**

   - Validan que los módulos integrados funcionen bien en conjunto y que nuevos cambios no rompan lo existente.
   - Detectar errores aquí ya es más costoso porque el sistema es más complejo.

7. **Pruebas de Criterios no Funcionales (rendimiento, seguridad, usabilidad, etc.)**

   - Se evalúan aspectos de calidad que no son funcionales.
   - Corregir fallas de rendimiento o seguridad en este punto puede requerir cambios estructurales grandes → alto costo.

8. **Pruebas de Aceptación del Usuario (UAT)**

   - El usuario final valida que el producto cumple los requerimientos.
   - Si encuentra fallas aquí, implican retrabajo significativo (ya se pasó todo el ciclo técnico).

9. **Criterios de Aceptación Operacionales (Producción)**

   - Es el nivel final, cuando el software ya está en producción.
   - Cualquier error detectado ahora tiene el máximo costo: afecta usuarios reales, operaciones, reputación, etc.

La flecha roja: "Costo del Error"

- Muestra que **cuanto más tarde se detecta un error, más caro es corregirlo**.

- Ejemplo:
    - Detectar un bug durante el unit test puede costar minutos.
    - Detectarlo en producción puede costar miles de pesos, pérdida de clientes y tiempo.